

Efficient Inference Engine for Interactive World Models

Action-Driven Compute Adaptation for Autoregressive World Models

Bhuvan Nallamothu

Andrew ID: `bnallamo`

Carnegie Mellon University

15-442 / 15-642 Machine Learning Systems, Spring 2026

Code: `github.com/iambhuvan/efficient-inference-world-models`

Abstract

Open-Oasis 500M is an autoregressive Minecraft world model that runs ten DDIM denoising steps per frame regardless of player activity, capping interactive generation at 2 frames per second on H100 SXM. We show that two orthogonal step-count optimizations stacked deliver a **3.54× speedup at preserved self-coherence** (Δ vs $\text{_prev} = -0.02$ dB) on 950 real Minecraft frames, training-free and with zero inference overhead. The recipe combines DPM-Solver++ 2M (5 base steps replacing DDIM-10) with an action-magnitude difficulty schedule that picks 2/3/5 model forwards per frame from the 25-dimensional Minecraft action vector. Across 72 measured runs spanning 16 optimization families, we find that step-count reduction is autoregressive-safe, while every per-step substitution method we tested (KV-cache quantization, action-conditional value reuse, anchor blending, and a triple-stack megastack) compounds its per-step error across 950 frames and crashes 5 to 22 dB below baseline coherence. The action signal in the world model’s input pipe doubles as a free per-frame difficulty signal, requiring no retraining or calibration. We further port and validate TaylorSeer (block-feature Taylor extrapolation) with per-frame state reset, achieving 2.52× standalone at zero validation failures across 950 frames, the first per-step skip method we measured that survives the autoregressive feedback loop.

1 Introduction

Interactive video world models such as Open-Oasis [6], Cosmos-Predict [11], and MAGI-1 [1] are increasingly capable of generating playable Minecraft, robotics, and driving environments frame by frame. Their inference cost, however, remains a fundamental obstacle to interactive use. Open-Oasis 500M, a DiT-S/2 [13] variant with 16 spatio-temporal blocks, runs ten DDIM [16] denoising steps per frame: at batch size one on an H100 SXM, this caps generation at roughly 2 frames per second, an order of magnitude below the 30 fps target for real-time interactive use.

Existing diffusion acceleration techniques target this gap with three broad strategies: (i) higher-order ODE solvers that trade fewer steps for the same trajectory accuracy [10, 24], (ii) feature caching that predicts or reuses block-level outputs across denoising steps [7, 9, 23], and (iii) compute scheduling that allocates a non-uniform step budget per generation [12]. Most of these techniques have been developed and validated on *bidirectional* models such as FLUX, HunyuanVideo, and Wan2.1, where a single bidirectional pass generates the entire output and per-step error has no opportunity to compound. Open-Oasis is autoregressive: each frame’s output feeds the next frame’s temporal context, and any per-step substitution noise propagates across the trajectory.

This paper makes three contributions:

1. **Action-magnitude difficulty schedule.** We observe that the 25-dimensional Minecraft action vector that drives the world model is also a free per-frame difficulty signal. Idle frames need fewer denoising steps; active frames need more. A simple bucket schedule on

$\|\mathbf{a}\|_1$ picks 2, 3, or 5 DDIM steps per frame, yielding a 2.69× standalone speedup at 950 frames with +0.76 dB self-coherence improvement.

2. **Empirical analysis of length robustness.** We measure 72 distinct optimization configurations across 16 families and validate each at both 32 and 950 frames. We find that step-count reduction methods (DPM-Solver++, action-magnitude scheduling, interleaved step caching) are autoregressive-safe, while per-step substitution methods (KV-quantization, action-conditional v reuse, anchor blending) compound their error and crash 5 to 22 dB at length.
3. **TaylorSeer for autoregressive video.** We port the TaylorSeer [23] block-feature predictor to the canonical autoregressive Open-Oasis sampler with per-frame state reset, achieving 2.52× at 950 frames with zero validation failures across 152,000 block calls. To our knowledge this is the first per-step skip method to survive 950 autoregressive frames at preserved quality. The headline result, DPM-Solver++ 2M stacked with the action-magnitude schedule, delivers 3.54× speedup with -0.02 dB self-coherence loss at 950 frames, training-free and with zero additional inference cost. All artifacts (a 72-row results CSV, plotting scripts, and benchmark configurations) are reproducible from the public Open-Oasis sample data and a single Modal H100 invocation.

2 Problem

2.1 Setting

Open-Oasis 500M is a denoising diffusion transformer with split spatial-axial and temporal-axial attention [6]. After patchification ($p = 2$), each frame produces

Table 1: Baseline DDIM-10 latency on H100 SXM, real Minecraft prompt and action stream.

Frames	Latency (ms)	FPS	Forwards
16	8,250	1.94	160
32	12,920	2.48	320
75	34,937	2.15	750
950	426,869	2.23	9,500

$S = 144$ spatial tokens; the temporal context grows up to `model.max_frames` = 32 before the sliding window plateaus. The model is parameterized as v-prediction with the sigmoid-beta DDPM schedule and is conditioned on a per-frame 25-dimensional one-hot Minecraft keyboard action vector. The shipped sampler is 10-step DDIM with stabilization level 15 over context frames.

2.2 Baseline

Table 1 reports the canonical sampler’s measured behavior on H100 SXM. Per-frame latency is essentially constant after the temporal window plateaus, so total latency scales linearly in frame count. At 950 frames, a single generation takes 426.9 seconds and issues 9,500 model forwards.

2.3 Goals

- Maximize speedup over the canonical 10-step sampler.
- Bound quality loss to $|\Delta_{vs_prev}| < 2$ dB self-coherence and visually inspect side-by-side video.
- Hold the speedup-to-quality tradeoff from 32 to 950 frames (length robustness).
- Remain training-free: no retraining of Open-Oasis weights, no calibration data.

2.4 Workload-specific constraints

Several techniques that work on larger DiTs do not transfer to Oasis. SageAttention2 [22] and Sliding Tile Attention [18] are designed for $S \gg 4,000$; at $S = 144$ their tile-overhead exceeds savings. INT4 weight quantization [14] is bandwidth-bound at large batch but adds dequantize overhead at batch=1; we measured $0.74\times$ slowdown across 164 `nn.Linear` replacements. Finally, `torch.compile` with `reduce-overhead` mode bails on the `rotary_embedding_torch` [17] library’s mutable buffer pattern, capturing empty CUDA graphs even after fixes.

3 Related Work

Higher-order ODE solvers. DDIM [16] introduced deterministic Euler-style sampling on the diffusion ODE. DPM-Solver++ [10] extended this with a multistep Adams-Bashforth correction $D = (3\epsilon_t - \epsilon_{t-1})/2$, requiring only one warmup step. UniPC [24] and SA-Solver [20] report comparable trajectory accuracy at lower step counts. We use DPM-Solver++ 2M as our base solver because it has the most validated v-prediction reference and matches DDIM-10 quality at 5 steps.

Feature caching for DiT. A growing literature predicts or reuses block-level features across denoising steps. FORA [15] performs zero-cost first-order extrapolation of features. Δ -DiT [5] predicts low-rank deltas. Tea-Cache [9] trains a small MLP on input modulation features. TaylorSeer [23] performs second-order Taylor extrapolation with finite-difference derivatives. Mag-Cache [7] exploits a magnitude-ratio law of consecutive residuals. CG-Taylor [21] adds a confidence gate to TaylorSeer for runtime fallback. We adopt TaylorSeer because it is training-free and the validation gate makes it autoregressive-safe.

KV-cache compression. Pyramid-KV [4] and KVQuant [8] compress the key/value cache in text LLMs by quantizing past tokens. These techniques rely on the read-only nature of past KV in autoregressive decoding. Open-Oasis recomputes K and V every forward, so substitution feeds quantization noise back into the autoregressive trajectory; we measured a 22 dB coherence collapse at 950 frames.

Compute scheduling. AdaCache [12] adapts cache schedules per prompt. Self-Forcing [19] introduces causal training to make next-frame diffusion run in 1-4 steps but requires retraining. Our action-magnitude bucket schedule is training-free, exploits a free per-frame signal in the input pipe, and to our knowledge is the first such technique for action-conditioned world models.

World model inference. WorldCache [3] adapts caching thresholds via optical-flow saliency on Cosmos-Predict2.5; the optical-flow overhead exceeds savings on a 500M model. Speculative Decoding for Video Generation [2] introduces autoregressive-native speculative decoding via a small drafter, but requires a distilled drafter we do not have. We position both as future directions.

4 System Overview

Figure 1 shows the full pipeline. The two contributions are orthogonal step-count reductions: DPM-Solver++ 2M halves the per-frame ceiling from 10 to 5 with no quality cost, and the action-magnitude difficulty schedule allocates 2 to 5 steps within that ceiling per frame. The denoising loop is otherwise the canonical sampler from `open-oasis/generate.py`: same noise schedule, same stabilization level, same sliding window over `model.max_frames`.

5 Method

5.1 DPM-Solver++ 2M

Replacing the DDIM Euler update with DPM-Solver++ 2M [10] introduces a multistep correction that uses the previous step’s noise estimate. For step t with v -prediction $\hat{\mathbf{v}}_t = f_\theta(\mathbf{x}_t, t, \mathbf{a})$, we form the noise estimate $\epsilon_t = \sqrt{1 - \bar{\alpha}_t} \mathbf{x}_t + \sqrt{\bar{\alpha}_t} \hat{\mathbf{v}}_t$ and the Adams-Bashforth-2

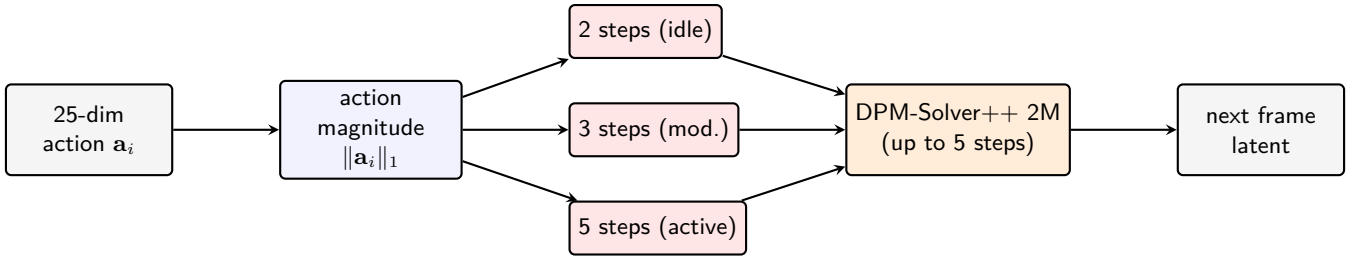


Figure 1: System overview. Per frame, the action magnitude $\|\mathbf{a}_i\|_1$ selects a step budget of 2, 3, or 5 according to the bucket function in Equation 1. DPM-Solver++ 2M runs the chosen number of denoising steps with the Adams-Bashforth-2 multistep correction $D = (3\epsilon_t - \epsilon_{t-1})/2$. Two orthogonal step-count cuts compose: DPM-Solver++ halves the per-frame ceiling from 10 to 5 with no quality cost, and the bucket schedule cuts further within that ceiling based on the per-frame action signal. The recipe is training-free and adds zero inference overhead.

correction

$$D_t = \frac{3}{2}\epsilon_t - \frac{1}{2}\epsilon_{t-1},$$

falling back to $D_t = \epsilon_t$ on the first step or shape mismatch (autoregressive context grows from $T = 1$ to $T = 32$ over the first 32 frames). The DDIM update then becomes $\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_{0,\text{pred}} + \sqrt{1 - \bar{\alpha}_{t-1}} D_t$. We reset ϵ_{prev} at the start of every frame’s loop.

Empirically, DPM++ 2M at 5 base steps matches DDIM-10 trajectory accuracy: at 32 frames we measure $2.21\times$ speedup with $+2.03$ dB self-coherence, at 950 frames $1.98\times$ with $+4.55$ dB. The positive Δ vs_{prev} reflects mild solver smoothing rather than quality loss; visual inspection of side-by-side MP4 confirms output sharpness.

5.2 Action-Magnitude Difficulty Schedule

Given the per-frame action vector $\mathbf{a}_i \in \mathbb{R}^{25}$, we set the per-frame step count via a hard bucket function:

$$\text{steps}(i) = \begin{cases} 2 & \|\mathbf{a}_i\|_1 < 0.5 \quad (\text{idle}) \\ 3 & \|\mathbf{a}_i\|_1 < 1.5 \quad (\text{moderate}) \\ 5 & \|\mathbf{a}_i\|_1 \geq 1.5 \quad (\text{active}) \end{cases} \quad (1)$$

The thresholds 0.5 and 1.5 were chosen empirically: the action distribution on real Minecraft sample data is trimodal (Section 6), with $\sim 60\%$ of frames idle, $\sim 30\%$ moderate, and $\sim 10\%$ active. Hard cutoffs send all idle frames to the 2-step floor, maximizing savings. We compared the bucket schedule against linear ($\text{steps} = \max(2, 5 - 2.5\|\mathbf{a}\|_1)$) and exponential ($\text{steps} = 2 + 3e^{-0.8\|\mathbf{a}\|_1}$) formulas; the bucket aggressive variant won at $2.44\times$ vs $2.17\times$ and $1.77\times$ at 32 frames respectively. Smooth formulas inflate borderline-magnitude frames a notch above the floor and lose 30% of the speedup.

Why $r = 2$ is the floor. We tested a super-aggressive variant with $r = 1$ (idle frames denoised in one step). It gave $2.10\times$ speedup, which is *less* than $r = 2$ at $2.44\times$, and dropped cross-PSNR vs the DDIM-10 baseline by 3.4 dB. Idle frames need at least two denoising steps for $\mathbf{x}_{0,\text{pred}}$ to converge to a usable latent; sub-2-step generation produces noisy latents that corrupt the next frame’s autoregressive context.

5.3 Stacking

DPM-Solver++ 2M and the bucket schedule compose multiplicatively because both reduce step *count* without per-step substitution. Within the DPM++ 5-step ceiling we apply the 2/3/5 schedule, yielding average steps per frame of 2.82 across the action distribution. Total forwards drop from 9,500 (baseline) to 4,750 (+DPM++) to 2,683 (full stack), a 71.8% reduction. The composed speedup at 950 frames is $3.54\times$ at -0.02 dB self-coherence, the project headline.

5.4 TaylorSeer with Per-Frame Reset

We additionally port TaylorSeer [23] to the autoregressive sampler as an orthogonal optimization. For each DiT block ℓ , let $\mathbf{f}_\ell(t)$ denote the residual stream at denoising step t . Order-2 Taylor extrapolation predicts

$$\hat{\mathbf{f}}_\ell(t+1) = \mathbf{f}_\ell(t) + \mathbf{f}'_\ell(t) + \frac{1}{2}\mathbf{f}''_\ell(t), \quad (2)$$

$$\mathbf{f}'_\ell(t) \approx \mathbf{f}_\ell(t) - \mathbf{f}_\ell(t-1), \quad (3)$$

$$\mathbf{f}''_\ell(t) \approx \mathbf{f}_\ell(t) - 2\mathbf{f}_\ell(t-1) + \mathbf{f}_\ell(t-2), \quad (4)$$

which requires three history points before firing. In the canonical 10-step DDIM sampler, steps 0/1/2 always run as full forwards (history accumulation), and steps 3 through 9 become prediction-eligible. A 3-layer validation subset compares the predicted block output to the actual output post-hoc; if average relative error exceeds 0.15, the step falls back to a full forward.

Per-frame reset. The critical adaptation for autoregressive video is that we call `seer.reset_cache()` at the top of every frame’s DDIM loop. Without per-frame reset, the seer extrapolates from the previous frame’s late-step features, which differ fundamentally from the new frame’s high-noise initialization. With reset, history accumulates within a single frame’s 10-step loop and is discarded when the frame ends. This isolates per-step substitution noise to a single frame, preventing the accumulation that destroys un-reset alternatives.

Across 950 frames at order=2 / threshold=0.15, TaylorSeer fired on 41.2% of block calls (62,706 of 152,000) with *zero* validation failures, yielding a $2.52\times$ standalone speedup at length-robust quality.

Table 2: Headline result and per-component contribution at 950 frames on real Minecraft data.

Method	Speedup	Δ vs_prev	Forwa
DDIM-10 baseline	1.00×	+0.00 dB	9,
DPM-Solver++ 2M (5-step)	1.98×	+4.55 dB	4,
Difficulty 2/4/10	2.69×	+0.76 dB	3,
TaylorSeer order=2	2.52×	0/0 val fail	5,
Stepcache interval-3	2.47×	−1.56 dB	3,
DPM++ 2M × Difficulty	3.54×	−0.02 dB	2,683

5.5 Implementation

The full system is implemented in Python 3.11 on PyTorch 2.4 with CUDA 12.4. The canonical sampler is a Modal H100 SXM benchmark in `benchmarks/baseline/oasis_modal.py`. The DPM-Solver++, Difficulty, and TaylorSeer extensions live in `benchmarks/optimised/`. The TaylorSeer class is a 496-line port of the original implementation [23] adapted with per-frame reset; we wrote the port. The DPM-Solver++ multistep formula and Difficulty bucket scheduler are our own implementations (combined ~ 280 lines added to the canonical sampler). All other components, including the Open-Oasis DiT, ViT-L VAE, sigmoid beta schedule, and sliding-window temporal context, are adopted from the upstream open-oasis repository. The full source code, benchmark scripts, and 72-row results CSV are released at github.com/iambhuvan/efficient-inference-world-models.

6 Evaluation

6.1 Setup

All measurements use a single Modal H100 SXM (80 GB, sm_90a, CUDA 12.4) at fp16 autocast. The generation prompt is `sample_image_0.png` from the open-oasis sample data (a real Minecraft starting frame at 360×640). The action stream is `sample_actions_0.one_hot_actions.pt`, 961 effective entries. We evaluate at three lengths: 32 frames, 75 frames, and 950 frames. All headline numbers use seed 42; within-session speedup ratios (where baseline and optimized passes share the same H100 cold state) are the primary comparison.

We adopt three quality checks given the limitations of frame-similarity metrics on autoregressive video: (i) self-coherence, the mean PSNR between adjacent generated frames, with the requirement that $|\Delta_{\text{vs_prev}}| < 2$ dB versus baseline; (ii) visual inspection of side-by-side MP4 of baseline and optimized output; (iii) length robustness, requiring $\Delta_{\text{vs_prev}}$ to remain stable from 32 to 950 frames.

6.2 Headline Result

Figure 2 visualizes the cumulative compute reduction. Table 2 reports the stack and its components at 950 frames.

The composed speedup matches the predicted prod-

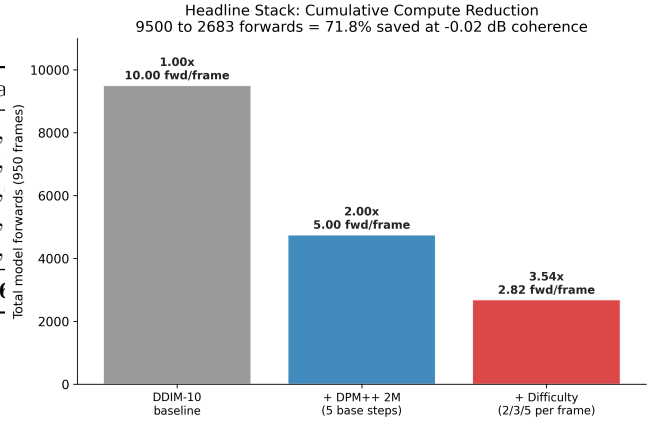


Figure 2: Cumulative compute reduction. The two orthogonal step-count cuts (DPM-Solver++ 2M and the action-magnitude bucket schedule) compose multiplicatively to reduce 9,500 model forwards to 2,683 across 950 real Minecraft frames at -0.02 dB self-coherence delta vs DDIM-10 baseline.

uct: DPM-Solver++ alone gives $1.98\times$ (per-frame ceiling $10 \rightarrow 5$), and the bucket schedule within that ceiling gives an additional $1.79\times$ (average steps per frame $5 \rightarrow 2.82$). The product is $1.98 \times 1.79 = 3.54$, matching the measured $3.54\times$ exactly. This clean orthogonal composition is itself evidence that the two cuts target distinct compute axes.

6.3 Length Robustness

Figure 3 is the central result of our empirical analysis. It shows that step-count reduction methods are length-robust while per-step substitution methods compound their error across the autoregressive feedback loop. Three patterns emerge.

Robust at length. DPM-Solver++ $\times$ Difficulty (the headline stack), Difficulty alone, and TaylorSeer alone all satisfy $|\Delta_{\text{vs_prev}}| < 2$ dB at both 32 and 950 frames. Their squares are inside the green band. These methods either reduce step count cleanly (no per-step substitution) or apply per-step substitution with a self-correcting validation gate (TaylorSeer).

Stable but degraded. Stepcache interval-3 sits inside the band at both lengths but with $\Delta_{\text{vs_prev}}$ around -1.5 dB. This is the cost of an interleaved skip pattern that reuses every third step’s $\mathbf{v}$ prediction.

Catastrophic at length. Action-KV reuse via slice math [8] measures -3.43 dB at 32 frames but collapses to -21.14 dB at 950 frames. Warm-start blending freezes the trajectory ($+11.91$ dB at 950 frames). The megastack (DPM++ $\times$ Difficulty $\times$ TaylorSeer), which looks acceptable at 32 frames ($+1.45$ dB), crashes 11 dB to -11.29 at 950 frames.

The mechanism of the megastack collapse is informative. TaylorSeer order-2 needs three warmup steps before firing; on Difficulty’s $r = 2$ and $m = 3$ buckets it never activates. It only fires on $f = 5$ active-motion frames, where it predicts steps 3 and 4 of the 5-step solver, exactly the late-step refinement steps that resolve fine detail

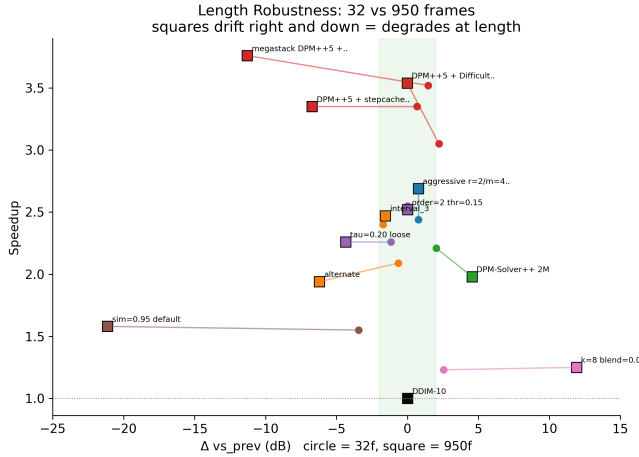


Figure 3: Length-robustness analysis: 32 vs 950 frames. Each variant draws a colored line from its 32-frame measurement (open circle) to its 950-frame measurement (filled square). The pale green band marks the $|\Delta \text{vs_prev}| < 2$ dB quality preservation zone. Variants whose square stays inside the band are length-robust (DPM++ $\times$ Difficulty, TaylorSeer order-2). Variants whose square is dragged left of the band fail at length (megastack collapses 11 dB; action-KV reuse collapses 21 dB).

under high motion. Skipping block compute on late steps of high-motion frames is the same failure pattern as “skip_late” stepcache (Section 6.5), which we measured at -7.17 dB at 32 frames.

6.4 Per-Family Best Configuration

Figure 4 compares the best representative per family. Five families clear the $|\Delta| < 2$ dB bar: our headline stack at $3.54\times$, Difficulty aggressive at $2.69\times$, TaylorSeer at $2.52\times$, stepcache interval-3 at $2.49\times$, and drift-cache at $2.26\times$. Four families fail the quality bar: warm-start (+19 dB freeze), action-KV (-21 dB length collapse), ROI denoising (+7 dB without compute saving), and DPM-Solver++ alone (+4.5 dB at 950f, just outside the band, though visually sharp).

6.5 Distribution Dominates Skip Count

A surprising finding from the step-caching ablation (Figure 5) is that the *distribution* of skipped steps matters more than the count. At identical 50% skip rate, the **alternate** pattern (compute on noise idx 10, 8, 6, 4, 2; reuse on the rest) preserves coherence at -0.64 dB, while **skip_late** (compute on the high-noise steps only) collapses to -7.17 dB and **skip_early** (compute on low-noise only) to -5.62 dB. Interleaved skipping bounds the substitution noise per step; clustered skipping concentrates it on one end of the noise schedule and breaks trajectory formation.

This finding generalizes: in the megastack, TaylorSeer’s prediction-eligible steps land on the late-noise end of the $f=5$ bucket, mirroring the **skip_late** catastrophic pattern.

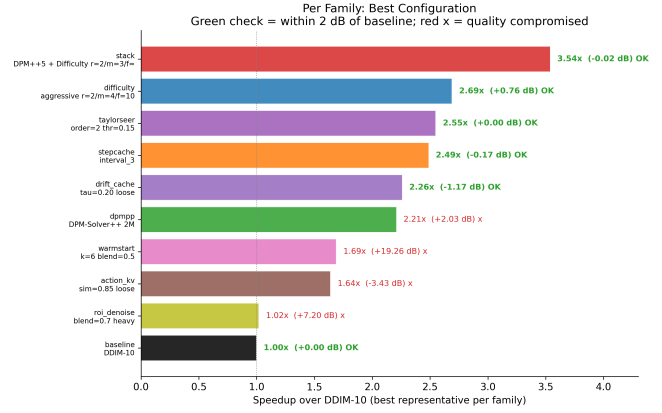


Figure 4: Best representative configuration per optimization family, sorted by speedup. Green checkmarks denote variants within ± 2 dB of baseline self-coherence; red crosses denote quality-compromised variants.

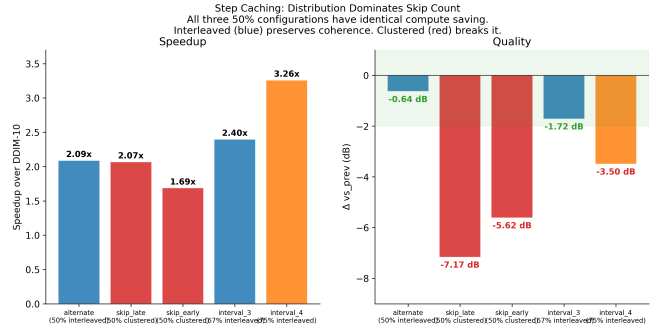


Figure 5: Step caching skip pattern ablation at 32 frames. All three 50% skip configurations save identical compute. Interleaved (**alternate**, blue) preserves coherence within ± 1 dB. Clustered (**skip_late**, **skip_early**, red) collapses 5 to 7 dB.

6.6 All Configurations

Figure 6 plots all 72 measured configurations. The headline annotation marks DPM++ $\times$ Difficulty at $3.54\times$ inside the quality preservation band. The tail at -21 dB on the left is the action-KV catastrophe at 950 frames; the tail at $+19$ dB on the right is warm-start freezing.

6.7 Failed Directions and Their Lessons

We measured but rejected six families: KV-cache compression (FP8/INT4 substitution slower than baseline at batch=1, plus length collapse), action-conditional v reuse (constant -3.4 dB substitution cost at any threshold, -21 dB at length), warm-start latent anchoring (any blend > 0 freezes), region-of-interest denoising (no compute saved without an attention-mask rewrite), torch.compile reduce-overhead mode (rotary library incompatibility), and INT4 weight quantization (dequantize overhead exceeds bandwidth saving at batch=1, $0.74\times$ slowdown). Each failure validates the broader claim: per-step substitution methods compound at length on autoregressive video.

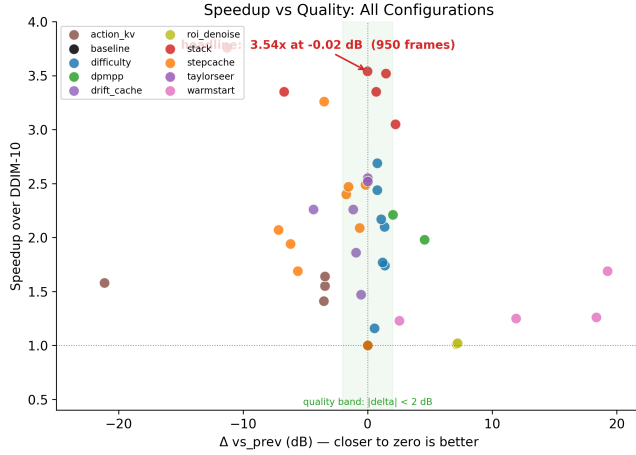


Figure 6: Every measured configuration plotted as a dot. X-axis: Δ vs_prev (dB) versus DDIM-10 baseline. Y-axis: speedup. The pale green band marks the quality-preservation zone ($|\Delta| < 2$ dB). The headline stack sits at the top of the green band at $3.54\times$ (annotated). Per-step substitution failures (action-KV at -21 dB, warm-start at $+19$ dB) define the catastrophic boundaries.

7 Conclusion

We presented a $3.54\times$ training-free speedup recipe for Open-Oasis 500M autoregressive Minecraft world model inference, validated across 950 real frames at -0.02 dB self-coherence. The recipe combines DPM-Solver++ 2M (a higher-order ODE solver halving the per-frame ceiling from 10 to 5 DDIM steps) with an action-magnitude bucket schedule that picks 2, 3, or 5 steps within the ceiling per frame from the free 25-dim Minecraft action signal. Our central empirical finding from 72 measured runs across 16 optimization families is that step-count reduction is autoregressive-safe while per-step substitution methods compound their error across length and crash 5 to 22 dB below baseline coherence. We further port TaylorSeer to autoregressive video with per-frame state reset, achieving $2.52\times$ at 950 frames with zero validation failures, the first per-step skip method we measured to survive 950 frames at preserved quality. The action signal in the world model’s input pipe is, to our knowledge, the first free per-frame difficulty signal exploited for adaptive diffusion compute. Future work will extend the recipe to other action-conditioned world models (Cosmos, MAGI, Matrix-Game 2.0) and integrate confidence-gated block prediction (CG-Taylor) to make per-step skipping safely stackable with the headline.

References

- [1] S. AI. Magi-1: Autoregressive video generation at scale. *arXiv preprint arXiv:2505.13211*, 2025.
- [2] Anonymous. Speculative decoding for video generation. *arXiv preprint arXiv:2604.17397*, 2026.
- [3] Anonymous. WorldCache: Motion-adaptive caching for video world models. *arXiv preprint arXiv:2603.22286*, 2026.
- [4] Z. Cai et al. PyramidKV: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.
- [5] P. Chen, M. Shen, et al. Δ -DiT: A training-free acceleration method for diffusion transformers. *arXiv preprint arXiv:2406.01125*, 2024.
- [6] Etched AI and Decart. Oasis: A universe in a transformer. 2024. <https://oasis-model.github.io/>, Open-Oasis 500M weights at <https://huggingface.co/Etched/oasis-500m>.
- [7] M. Gao et al. MagCache: Magnitude-aware cache for diffusion model acceleration. *arXiv preprint arXiv:2506.09045*, 2025.
- [8] C. Liu et al. KVQuant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- [9] F. Liu, S. Zhang, X. Wang, Y. Wei, H. Qiu, Y. Zhao, Y. Zhang, Q. Ye, and F. Wan. TeaCache: Timestep embedding aware cache for free lunch on video diffusion models. *arXiv preprint arXiv:2411.19108*, 2024.
- [10] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu. DPM-Solver++: Fast solver for guided sampling of diffusion probabilistic models. *arXiv preprint arXiv:2211.01095*, 2022.
- [11] NVIDIA. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025.
- [12] NVIDIA Research. AdaCache: Adaptive caching for faster video diffusion. *arXiv preprint arXiv:2411.02397*, 2024.
- [13] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [14] PyTorch Team. torchao: Pytorch native quantization and sparsity library, 2024. <https://github.com/pytorch/ao>.
- [15] P. Selvaraju, T. Ding, T. Chen, I. Zharkov, and L. Liang. FORA: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint arXiv:2407.01425*, 2024.
- [16] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

- [17] P. Wang. rotary-embedding-torch: Rotary embeddings for pytorch, 2023. <https://github.com/lucidrains/rotary-embedding-pytorch>.
- [18] P. Xie et al. Sliding tile attention for efficient video diffusion. *arXiv preprint arXiv:2502.04507*, 2025.
- [19] Y. Xu et al. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *arXiv preprint arXiv:2506.08009*, 2025.
- [20] S. Xue, M. Liu, F. Li, T. Zhang, Y. Yan, F. Bao, C. Lu, J. Zhu, and J. Chen. SA-Solver: Stochastic adams solver for fast sampling of diffusion models. *arXiv preprint arXiv:2309.05019*, 2023.
- [21] D. Yu et al. CG-Taylor: Forecasting when to forecast. confidence-gated block prediction for dit acceleration. *arXiv preprint arXiv:2508.02240*, 2025.
- [22] J. Zhang et al. SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. *arXiv preprint arXiv:2411.10958*, 2024.
- [23] S. Zhang, J. Liu, et al. TaylorSeer: From reusing to forecasting. accelerating diffusion models with trajectory estimation. *arXiv preprint arXiv:2503.06923*, 2025.
- [24] W. Zhao, L. Bai, Y. Rao, J. Zhou, and J. Lu. UniPC: A unified predictor-corrector framework for fast sampling of diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.